Behavioral Fingerprinting: Monitoring LLM Agent Stability Across Model Upgrades

Sathiyan Kutty
Independent Researcher

March 2026

Abstract

Large language model (LLM) powered agents are increasingly deployed in production environments to perform coding, retrieval, analysis, and workflow automation tasks. However, upgrading the underlying model can introduce subtle behavioral drift even when benchmark performance appears to improve. Such drift may change reasoning depth, tool-selection behavior, output formatting, or security posture, thereby destabilizing downstream systems. This paper introduces *Behavioral Fingerprinting*, a telemetry-based framework for monitoring agent stability across model upgrades. Rather than evaluating final-answer correctness alone, the method represents agent behavior as a vector of execution features derived from agent traces, including reasoning depth, tool invocation probability, output length, token-level uncertainty proxies, and self-correction frequency. We further introduce two complementary evaluation techniques: *Prompt Invariance Testing*, which measures stability under semantically equivalent prompts, and *Agent State Replay*, which re-executes historical agent workflows under candidate model versions. We argue that model upgrades should be treated as behavioral and security migrations, not merely capability upgrades, and present a practical framework for deployment-time drift detection in enterprise AI systems.

1 Introduction

Large language models have rapidly become the cognitive core of modern software agents. In production systems, these agents often perform multi-step tasks involving planning, retrieval, tool use, code generation, execution, and interaction with enterprise data systems. As vendors release new model versions, organizations are incentivized to upgrade in order to gain better reasoning, lower latency, lower cost, or improved coding performance.

However, an LLM upgrade is not equivalent to a standard software patch. Even when the new model appears superior on public benchmarks, its operational behavior inside an agent can change in ways that are difficult to predict and dangerous to ignore. A candidate model may call tools more aggressively, follow embedded instructions more literally, produce differently structured outputs, or become more sensitive to semantically irrelevant prompt changes. These differences can cause cascading failures in production workflows.

This paper focuses on *behavioral drift*: changes in agent decision dynamics induced by model upgrades. Existing evaluation methods in the LLM literature primarily emphasize task accuracy, benchmark performance, or preference alignment. These are important but insufficient. In production agent systems, output correctness alone does not guarantee workflow stability.

Behavioral drift occurs when the internal reasoning dynamics of an agent change across model versions, even if the final outputs appear correct. Public production evidence suggests that such

drift is not merely theoretical: model migrations have been reported to reduce prompt-injection resistance, alter tool-use behavior, and force teams to retain legacy models alongside newer ones to preserve stability.

We propose *Behavioral Fingerprinting*, a framework that models agent behavior as a telemetry-derived vector and measures divergence across model versions. The paper makes three contributions:

1. **Behavioral Fingerprinting**: a vector representation of agent decision dynamics suitable for drift monitoring.
2. **Prompt Invariance Testing**: a fuzz-testing approach for measuring behavioral stability under semantically equivalent prompts.
3. **Agent State Replay**: a method for replaying previously observed workflows under a new model to detect execution-path divergence before deployment.

Together, these techniques provide a practical blueprint for organizations seeking to upgrade agentic systems without silently degrading reliability or safety.

2 Background and Related Work

Research on language-model evaluation has largely focused on correctness, broad capability measurement, and alignment. HELM introduced a standardized framework for evaluating language models across multiple axes such as accuracy, robustness, efficiency, and calibration [**?**]. Work on chain-of-thought reasoning and self-consistency demonstrated that inference-time reasoning trajectories materially affect performance on multi-step problems [**?**, **?**]. Alignment methods such as reinforcement learning from human feedback and constitutional training focus on steering behavior toward preferred outputs or policy-compliant completions [**?**, **?**].

At the systems level, recent literature has begun to study LLM agents, tool use, and production architectures. However, there is still relatively limited formal treatment of the specific problem addressed here: ensuring that an agent remains behaviorally stable when the underlying model changes. In deployed environments, engineering teams frequently discover that a model upgrade changes message formatting, tool-calling tendencies, safety posture, or prompt sensitivity, even when top-line benchmark scores improve.

This gap motivates a distinct evaluation objective. We are not asking only whether a new model is "better" in aggregate. We are asking whether a system built on top of that model remains acceptably stable, predictable, and safe after migration.

3 Problem Definition: Agent Behavioral Drift

Consider an agent system composed of a language model M, a tool set $\mathcal{T}$, an execution policy Π, and mutable state S. Given user input x, the system produces an execution trace

$$\tau(M, x, S) = (a_1, a_2, \ldots, a_n), \tag{1}$$

where each action a_i may represent an internal reasoning step, a tool invocation, a message-formatting choice, a memory update, or a final response.

If the system is upgraded from model M_1 to model M_2, then in general

$$\tau(M_1, x, S) \neq \tau(M_2, x, S). \tag{2}$$

This divergence does not necessarily imply a failure. A new model may discover a shorter or more accurate reasoning path. However, a production failure can emerge when the divergence violates operational expectations, such as:

- calling a tool when the baseline model would not,
- failing to call a required retrieval or verification tool,
- generating a differently structured output that breaks a parser,
- exhibiting degraded resistance to prompt injection or adversarial context,
- increasing verbosity or uncertainty in a way that affects downstream orchestration.

We therefore define *behavioral drift* as the divergence in operationally relevant agent behavior induced by a model upgrade:

$$D_{\text{behavior}}(M_1, M_2; x, S) > \delta, \tag{3}$$

where δ is an application-specific tolerance threshold.

4 Behavioral Fingerprinting

We represent agent behavior as a fingerprint vector extracted from execution telemetry:

$$F = [R, T, L, H, C, P, G], \tag{4}$$

where:

- R is average reasoning depth,
- T is tool invocation probability,
- L is output length distribution,
- H is a token-level uncertainty proxy or entropy estimate,
- C is self-correction frequency,
- P is parser-compatibility or schema-conformance rate,
- G is tool-gating compliance or policy-conformance rate.

A simple aggregate fingerprint over a workload $\mathcal{X}$ may be written as

$$F(M; \mathcal{X}) = \frac{1}{|\mathcal{X}|} \sum_{x \in \mathcal{X}} \phi(\tau(M, x, S_x)), \tag{5}$$

where $\phi(\cdot)$ extracts telemetry features from an execution trace.

The drift score between two model versions is then

$$\Delta(M_1, M_2; \mathcal{X}) = \|W \odot (F(M_1; \mathcal{X}) - F(M_2; \mathcal{X}))\|_2, \tag{6}$$

where W is a vector of application-specific feature weights and $\odot$ denotes elementwise multiplication.

This weighting is important. For a code-generation agent, parser conformance and tool-calling behavior may matter more than verbosity. For a retrieval-augmented safety-sensitive agent, prompt-injection resistance and tool-gating compliance may dominate.

4.1 Operational Interpretation

Behavioral Fingerprinting is intentionally not a pure semantic-quality metric. Two models may produce identical final answers while exhibiting materially different risk profiles. For example:

- Model M_1 retrieves supporting evidence before answering.
- Model M_2 answers directly from priors without retrieval.

If both answers appear correct on a shallow eval set, standard benchmarking may miss the fact that M_2 is operationally less trustworthy. Fingerprinting surfaces that difference.

5 Prompt Invariance Testing

A stable agent should not exhibit large behavioral changes under semantically equivalent prompts that preserve the same task intent. We therefore define a prompt-invariance set

$$\mathcal{P}(x) = \{p_1, p_2, \ldots, p_k\}, \tag{7}$$

where each p_i encodes the same intended task as base prompt x.

Example prompt variants might include:

"Summarize this document."
"Provide a short summary of this document."
"Give a concise overview of the document."

For a given model M, we measure intra-set variance in behavioral fingerprints:

$$V_{\text{inv}}(M, x) = \frac{1}{k} \sum_{i=1}^{k} \left\| F(M; p_i) - \bar{F}(M; \mathcal{P}(x)) \right\|_2, \tag{8}$$

where $\bar{F}(M; \mathcal{P}(x))$ is the mean fingerprint across the prompt set.

High prompt-invariance variance indicates that the agent is overly sensitive to superficial prompt wording. This is especially relevant in production environments where prompts may be templated, transformed, localized, or partially generated by upstream systems.

6 Agent State Replay

Historical workflows contain valuable information about what a production system actually experiences. We propose *Agent State Replay* as a migration-time evaluation method in which past traces are re-executed under a candidate model.

A replay record contains:

$$\rho = (x, S_0, y_{tool}, \tau_{baseline}), \tag{9}$$

where x is the original input, S_0 is the initial agent state, y_{tool} represents observed tool outputs or controllable mocks, and $\tau_{baseline}$ is the baseline execution trace.

The replay procedure is:

1. Load the historical prompt and state.
2. Replay tool outputs deterministically or through mocks.

3. Execute the same workflow using candidate model M_2.
4. Compare the new trace $\tau(M_2, x, S_0)$ to baseline trace $\tau_{baseline}$.

This yields an execution-path divergence score such as

$$D_{\text{trace}} = d(\tau_{baseline}, \tau(M_2, x, S_0)), \tag{10}$$

where $d(\cdot, \cdot)$ may be implemented via edit distance over action types, graph distance over execution DAGs, or a weighted mismatch score over tool calls, schemas, and policy decisions.

7 Real-World Case Study: Safety Regression After a Model Upgrade

A publicly documented example comes from Promptfoo, which reported testing a customer's agent after an upgrade from GPT-4o to GPT-4.1 [**?**]. In their evaluation harness, the agent's prompt-injection resistance dropped from 94% to 71% after the migration. The regression occurred even though the newer model demonstrated improved instruction-following in other contexts.

Promptfoo attributes the regression to the upgraded model following embedded instructions more literally, including instructions contained within retrieved documents. While this behavior improved certain reasoning tasks, it increased the agent's vulnerability to indirect prompt-injection attacks delivered through external data sources.

Mitigation required changes beyond simply adopting the new model. The engineering team introduced multiple system-level safeguards: an output classifier to detect malicious instructions, stricter tool invocation gating, and revised system prompts tailored to the new model behavior. This case highlights a critical property of agent systems: model upgrades can alter operational risk profiles, not merely output quality. In agentic workflows, subtle changes in reasoning behavior can influence which tools are called, how external information is interpreted, and whether security policies are followed.

This incident aligns with broader production observations. A 2025 study of deployed agents reports that engineering teams frequently run legacy and newer models in parallel because agent scaffolds depend on model-specific behavior; sudden migrations can introduce behavioral changes that degrade output quality or destabilize workflows [**?**]. Vendor migration documentation reinforces this point. Model upgrade guides warn that newer model versions may introduce breaking changes in areas such as tool-call formatting, assistant message structure, argument serialization, and instruction-following behavior [**?**]. Such changes can break production pipelines even when the new model demonstrates superior benchmark performance.

Taken together, these observations suggest that model upgrades should be treated as behavioral and security migrations, not simply capability improvements. This motivates Behavioral Fingerprinting: migrations should be evaluated not only for task accuracy but also for stability in tool use, prompt-injection resistance, schema behavior, and execution-path dynamics.

8 Evaluation Methodology

A practical evaluation suite for Behavioral Fingerprinting should combine offline traces, synthetic adversarial cases, and workflow-specific acceptance tests. We propose the following categories:

8.1 Trace-Based Regression Set

Construct a corpus of historical production or staging traces spanning representative workflows. Compute baseline fingerprints under the incumbent model and compare against candidate models.

8.2 Prompt Invariance Set

Create semantically equivalent prompt bundles for each critical task. Measure both output-level consistency and behavioral variance.

8.3 Security and Tooling Set

For agents that use retrieval or external tools, include cases involving prompt injection, tool misuse, malformed arguments, schema drift, and refusal-policy boundaries.

8.4 Acceptance Criteria

A deployment should not be approved solely because benchmark accuracy increases. Instead, organizations can set thresholds such as:

$$\Delta(M_1, M_2; \mathcal{X}) \leq \delta_F, \tag{11}$$

$$D_{\text{trace}} \leq \delta_\tau, \tag{12}$$

$$V_{\text{inv}}(M_2, x) \leq \delta_P \quad \forall x \in \mathcal{X}_{critical}. \tag{13}$$

These can be combined with traditional quality metrics into a composite migration score:

$$S_{\text{mig}} = \alpha Q - \beta \Delta - \gamma D_{\text{trace}} - \eta V_{\text{inv}}, \tag{14}$$

where Q represents task-quality improvement and the remaining terms penalize instability.

9 Discussion

Behavioral Fingerprinting is useful because it reframes model migration as a systems problem rather than a pure-model problem. In practice, organizations do not deploy models in isolation. They deploy orchestrated workflows whose reliability depends on stable interactions among prompts, tools, parsers, policy layers, and memory.

The framework also supports gray areas rather than forcing a simplistic pass-or-fail interpretation. Some behavioral drift is desirable. A newer model may reduce unnecessary tool calls, shorten reasoning paths, or improve instruction-following in ways that benefit the system. The objective is not zero drift. The objective is to detect and bound drift that is operationally harmful.

This distinction matters because capability improvements and workflow stability can move in different directions. A model that scores better on general benchmarks may still be a worse fit for a particular agent scaffold. Conversely, a model that appears slightly weaker on broad benchmarks may preserve invariants that make it more production-worthy.

10 Limitations

This work has several limitations. First, some fingerprint features depend on telemetry that may not be exposed by all model providers. Second, feature choice is application dependent; there is

no universal fingerprint that fits every agent architecture. Third, certain forms of behavioral drift may be visible only under long-horizon or rare workflows that are underrepresented in historical traces. Finally, this paper proposes a framework and measurement strategy rather than presenting a large-scale empirical benchmark suite.

These limitations do not invalidate the approach, but they do suggest that future work should focus on standardized trace datasets, cross-provider telemetry conventions, and broader validation across agent frameworks.

11 Conclusion

As LLM-powered agents become central to production software systems, model upgrades create a new operational challenge: behavioral drift. Traditional evaluation methods focus mainly on correctness, preferences, or benchmark capability, leaving a gap in deployment-time stability analysis.

This paper introduced Behavioral Fingerprinting, a telemetry-based framework for monitoring agent stability across model upgrades. By modeling behavior through execution-level features, and combining fingerprint divergence with prompt invariance testing and agent state replay, the framework provides a practical method for detecting migration risk before production rollout.

The broader implication is straightforward: model upgrades should be treated as behavioral and security migrations. Doing so creates a more realistic, engineering-grounded path to reliable enterprise AI deployment.

Future Work

Future work should include standardized behavioral-trace corpora, open-source fingerprint extraction libraries, benchmark suites covering retrieval, coding, and API-orchestration agents, and empirical studies of which fingerprint features best predict production incidents.

References

[1] Percy Liang, Rishi Bommasani, Tony Lee, Dmitriy Turbiner, et al. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research*, 2023. https://arxiv.org/abs/2211.09110

[2] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *NeurIPS*, 2022. https://arxiv.org/abs/2201.11903

[3] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR*, 2023. https://arxiv.org/abs/2203.11171

[4] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, et al. Training Language Models to Follow Instructions with Human Feedback. *NeurIPS*, 2022. https://arxiv.org/abs/2203.02155

[5] Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, et al. Constitutional AI: Harmlessness from AI Feedback. 2022. https://arxiv.org/abs/2212.08073

[6] Promptfoo. Model Upgrades Break Agent Safety. Blog post, 2026. https://www.promptfoo.dev/blog/model-upgrades-break-agent-safety/

[7] Anonymous / public preprint source. Production Agents Study. 2025. https://arxiv.org/html/2512.04123v1

[8] Anthropic. Migrating to Claude 4. Documentation, 2026. https://docs.anthropic.com/en/docs/about-claude/models/migrating-to-claude-4